GLP Networking API Specification

Dan Bachar
Technical University of Munich
and Technion–Israel Institute of Technology

Ehud Shapiro
London School of Economics
and Weizmann Institute of Science

Abstract

Multiagent deterministic GLP (madGLP) [8] runs Grassroots Logic Programs (GLP) [6] as agents that communicate solely by asynchronous message passing, and assumes fair delivery: every message sent is eventually delivered. Each agent runs on a person's smartphone and is named by the person's public key. A grassroots platform depends on no global resource other than the network [5, 7], so the agents must reach one another face to face over Bluetooth Low Energy, on a local network, and across the Internet from behind NAT, with rendezvous servers that anyone can run for connection establishment. What madGLP requires of such a network, and what the network requires of the platform above it, has not been specified.

Here, we specify the networking layer beneath madGLP. The guiding principle is that anything that can be done in GLP is done in GLP: the layer provides what GLP cannot reach by itself — the network stack, the platform's location service, the platform's attestation of the application it runs, and cryptographic primitives — and everything above that line is GLP code: the social graph, friendship, introduction, the distribution of addresses and the reconnection of peers whose address changed, and the rendezvous protocol.

We specify three transport underlay realizations, all following one interface. Over the PAN and LAN transport underlays, an agent advertises a rotating recognition token derived from its key and the current time slot, so that a peer who knows the key recognises it before any contact and nobody else can track it; whether an agent answers an unrecognised contact is a trust level GLP sets per underlay, raised on entering a place GLP declared and lowered on leaving it. Over the WAN underlay, a rendezvous server is a host with a static address running the headless layer and a GLP rendezvous agent, which observes addresses and coordinates hole punching, relays no payload, and belongs to no social graph. On every underlay, the peers establish a mutually authenticated Noise session from their Ed25519 keys, IK when the initiator already knows the responder's key and XX only for a first contact under Open trust, and then attest to each other, per session and bound to the session's handshake hash, that each runs the application published under its developer's key, verified against Apple's and Google's public roots with no third party contacted during a session; we state what each platform's attestation proves and what it does not, and the runtime reports to the program whether a peer is attested. One message protocol carries payloads on every underlay, over UDP datagrams and over GATT writes, with fair, unordered, atomic and authenticated delivery, which is what madGLP assumes.

1 Introduction

Multiagent deterministic GLP (madGLP) [8] is a distributed runtime for Grassroots Logic Programs (GLP) [6] in which autonomous agents communicate solely by message passing, each agent on a person's smartphone and named by the person's public key. This document specifies the networking layer beneath madGLP: the interface that enables madGLP agents to run on separate devices communicating peer-to-peer, and the layer functions behind the system predicates by which GLP programs reach it. madGLP and the runtime's side of the seam are specified in the implementation paper [8]; the seam predicates are in the language paper's catalogue [6]; the operating system above the runtime, the Grassroots Super-App with its social graph (in preparation), is what decides whom to befriend and what an attested or unattested peer may be used for; this paper specifies every service below GLP that the super-app needs, the rendezvous server with its GLP agent among them (Section 6).

Layering. The guiding principle is that anything that *can* be done in GLP is done in GLP: the networking layer provides only what GLP cannot reach by itself—the network stack (PAN, LAN and WAN), the platform's location service (geofencing), the platform's remote attestation of the application it runs, and cryptographic primitives (Ed25519 signing and verification, the Noise handshake). Everything above this line is GLP agent code. In particular, the social graph [7], friendship, introduction and the distribution of addresses are implemented in GLP above this specification, and so are the rendezvous agent and the reconnection and invitation protocols that run through it, which Section 6 specifies with the server they run on; to the networking layer, all madGLP messages are opaque payloads. The interface between the two consists of the API defined here together with five system predicates of the language [6], whose layer functions Section 5 specifies.

One overlay, three underlays. The API is underlay-agnostic at its core: madGLP issues the same calls regardless of how a message travels. Three underlays realize the core, the overlay being the one address space of public keys over them:

- **PAN** (Bluetooth Low Energy, Section 3.3): communication between devices in direct radio range, enabling offline, infrastructure-free operation. The PAN is the bootstrap underlay where agents first meet face-to-face.
- **LAN** (a local network, Section 3.2): communication between devices on the same local network, found by DNS-SD over multicast DNS and carried over IP.
- **WAN** (Internet Protocol, Section 4): communication between remote devices at arbitrary distances, using UDP datagrams, with rendezvous servers for signaling and NAT traversal.

Some control operations are inherently underlay-specific—observing the agent's own public address is IP-only—and are specified with their underlay rather than in the agnostic core, and called on that underlay's sub-object (Section 2.2). PAN and LAN discovery—finding agents within radio range or on the same local network—spans both underlays.

Outline. Section 2 specifies the underlay-agnostic API the madGLP runtime calls. Section 3 specifies proximity PAN/LAN-based discovery and communication: its principles, then its realizations. Section 4 specifies the WAN realization. Section 5 specifies the layer functions behind the five system predicates through which GLP agent code reaches the layer. Section 6 specifies the rendezvous server, its networking requirements and its GLP agent with the reconnection and invitation protocols. Section 7 summarizes the complete API and its correspondence to madGLP.

2 Networking API

This section specifies the underlay-agnostic interface between the madGLP runtime and the networking layer. The madGLP runtime issues these calls without regard to how a message travels; the PAN (Section 3.3), LAN (Section 3.2) and WAN (Section 4) realizations carry them.

Throughout, a peer is named by its public key. An `Underlay` is one of `PAN`, `LAN` or `WAN`; a peer may be reachable over more than one underlay at a time. The *proximity underlays* are `PAN` and `LAN`, on which a nearby agent is discovered (Section 3). A LAN connection and a WAN connection are both IP connections; the layer tells them apart by whether the peer's address is on the agent's own local network (Section 2.8). An IP connection is a `LAN` connection when the layer runs `LAN` and the peer's address is local; otherwise it is a `WAN` connection when the layer runs `WAN`; otherwise the layer neither makes nor accepts it.

The underlays are independent realizations: an agent may implement any subset of them and remains a conforming agent—identity, known peers, sessions, point-to-point delivery, trust, and the networking assumptions (Section 2.11) are unchanged, and an unimplemented underlay is simply absent from discovery and `peerUnderlays`, and has no sub-object (Section 2.2). In particular, a server-side agent—a rendezvous server (Section 6), or a service co-hosted with one—realizes the WAN underlay, headless.

2.1 Identity

Each agent has an Ed25519 key pair. The 32-byte public key is the agent's globally unique identity, used for addressing, authentication, and signing, and is persistent across sessions. The layer holds the private key, signs with it and authenticates the sender of every message, and is part of the application the platform attests (Section 2.6). The key pair is generated and persisted above the networking layer and is supplied to the layer's constructor when it is instantiated; it is immutable for the layer's lifetime. Key rotation or a move to another device is realized by instantiating the layer with another pair.

Function	Signature	Description
Retrieve identity	`getIdentity()` → `PubKey`	Return the agent's public key.

2.2 Instantiation

The layer is created once per agent, with everything it needs to run, and its parameters hold for its lifetime.

Parameter	Type	Description
Key pair	Ed25519 key pair	The agent's identity (Section 2.1).
Underlays	`Underlay[]`	The underlays the layer runs: a subset of `PAN`, `LAN`, `WAN`; a headless server runs `WAN` (Section 2).
Server address	`Address?`	A rendezvous server's listening address (Section 6); null on a smartphone.
Rendezvous server	boolean	Whether the agent is a rendezvous server (Section 6).

The layer exposes one sub-object for each underlay it runs, `pan`, `lan` and `wan`, and null for an underlay it does not run. The functions of this section are called on the layer itself, as they span all underlays: the layer chooses the underlay that carries each message, and reports reachability over all of them, so the callbacks that concern a peer carry the `Underlay` they refer to. The functions that belong to one underlay are called on its sub-object, and take no underlay parameter: discovery and the trust level on `pan` and `lan` (Section 3.1), and the public address, peer addresses, hole punching, the rendezvous servers and the local network identity on `wan` (Section 4). Thus, a call that makes no sense on an underlay, for example a trust level on the WAN, cannot be written, and a call on an underlay the layer does not run finds no sub-object.

2.3 Connection and Reachability

The networking layer notifies the runtime when peers become reachable or unreachable, over any underlay, and reports the underlay(s) by which a peer is currently reachable. Reporting the underlay lets GLP make proximity-aware decisions—for example, GLP computes which of its friends are physically nearby by intersecting its friends list with the peers currently reachable over `PAN`. Which underlay carries a given message, and which path or rendezvous server is used, remain the networking layer's choice. The connection report says whether the peer's identity is attested: it carries the attested application, or `null` for an unattested identity (Section 2.6), and the runtime reports it to the program [8]; what an unattested peer may be used for is GLP's decision.

Function	Signature	Description
Peer connected	`onPeerConnected(pk, underlay, application?)`	Callback when a peer becomes reachable for communication. Receives the peer's public key, the `Underlay` over which it connected, and, for an attested identity, the attested application: its developer, its name and its version number; `null` for an unattested identity (Section 2.6).
Peer disconnected	`onPeerDisconnected(pk, underlay)`	Callback when a peer is no longer reachable over a given `Underlay`. Receives the peer's public key and that underlay.
Check reachability	`isPeerReachable(pk)` → `bool`	Whether the given peer is currently reachable via any underlay.
Reachable underlays	`peerUnderlays(pk)` → `List<Underlay>`	The underlays by which the given peer is currently reachable (empty if unreachable).

2.4 Known Peers

The layer holds no social graph. It holds a *known-peer set*: the public keys GLP has supplied, each optionally carrying a last recorded address. This set is what Closed trust (Section 2.8) recognizes, what unsolicited inbound WAN contact is checked against, what reconnection attempts range over, and where `putPeerAddress` (Section 4.2) records addresses. Whether a known peer is a friend is GLP's interpretation; removing a key removes the layer's recognition of it.

Function	Signature	Description
Add known peer	`putKnownPeer(pk)`	Add the public key to the known-peer set.
Remove known peer	`removeKnownPeer(pk)`	Remove the key and any recorded address from the known-peer set.

2.5 ANNOUNCE and Liveness

ANNOUNCE is the protocol's identity beacon and heartbeat. It is a self-contained signed record whose payload carries:

- the sender's full 32-byte public key;
- the protocol version (2 bytes);
- a trailing Ed25519 signature over all preceding bytes.

The signature is verified against the carried sender key before the record is accepted, and repeated ANNOUNCEs from the same key are idempotent.

Role per underlay. ANNOUNCE serves a different mix of purposes on each underlay:

- **Over the PAN** it is, first, the *identity bootstrap* of a first contact under Open trust: a scan match of an unrecognized presence carries no public key (Section 3.3.2), so ANNOUNCE supplies the key that the Noise XX handshake that follows checks (Section 2.6). A recognized presence needs no ANNOUNCE before the handshake, as its token already names the key the IK handshake expects. Second, it is a *liveness check*: link death is normally reported by the link layer itself, and the announce-driven sweep covers the cases where the link layer reports no disconnect. The staleness clock counts any authenticated traffic; ANNOUNCE guarantees a floor of traffic on an idle link. When this check marks a connection as stale, the agent severs both GATT legs before synthesizing the disconnect.
- **Over LAN and WAN** it is the *liveness signal*: a UDP path has no connection state; a session silent for two consecutive intervals is torn down, feeding `onPeerDisconnected` (Section 2.3). A WAN connection is initiated by address with the peer's key already expected (Section 2.6); on the LAN, ANNOUNCE bootstraps identity only for a first contact under Open trust, as on the PAN. An inbound connection is identified by its completed handshake.

Heartbeat. Once the session is established, ANNOUNCE is sent inside it, and repeats periodically (every 10 s in the implementation) on every live connection of every underlay; only the ANNOUNCE of a first contact under Open trust travels before the handshake. A peer silent for two consecutive intervals is stale *on that underlay*, independently per underlay, with the per underlay consequences above. The same heartbeat tick drives reconnection attempts towards unreachable known peers and rendezvous servers and the replay of queued messages once a recipient is live again (fair delivery, Section 2.11).

Trust gating. Under Closed trust on a proximity underlay (Section 2.8), no ANNOUNCE is sent or answered before the handshake: the agent contacts a recognized peer with the IK handshake, and an ANNOUNCE arriving before a handshake is ignored and its connection dropped. Unsolicited WAN contact is gated by the known-peer set instead (Section 2.8). Peer addresses reach the layer via `putPeerAddress` (Section 4.2), fed by GLP. Over LAN and WAN every connected peer is already authenticated.

2.6 Session Establishment

Once a transport-level connection to a peer exists—a GATT connection on the PAN (Section 3.3) or a UDP path on the LAN or WAN (Section 4)—the peers perform a mutual authentication handshake using their Ed25519 keys via Noise (Curve25519 for key exchange, ChaCha20-Poly1305 for encryption, SHA-256 for hashing). The handshake proves that each peer holds the private key of its static public key, and derives forward-secret session keys. The Noise pattern depends on what the initiator already knows. When the initiator expects a specific responder key, it runs Noise IK: it sends its own static key in the first message, encrypted to the expected key, so only the holder of that key can read or answer it, and the responder never transmits its static key. The initiator expects a key for a known peer (Section 2.4), for a recognized proximity token (Section 3.1), for a rendezvous server, whose key GLP declared (Section 4.2), and on every WAN connection, since the layer connects over the WAN only to known peers (Section 4.4). Only a first contact with an unrecognized presence under Open trust (Section 2.8) has no expected key: there the peers exchange ANNOUNCE first (Section 2.5) and run Noise XX, in which each side transmits its static key, and each verifies the delivered key against the key ANNOUNCE claimed; a mismatch aborts the connection. The responder enforces the trust level: under Closed trust, and on the WAN except on a rendezvous server, it answers an IK first message only from a key in the known-peer set, and answers no XX first message, so a stranger never learns its key. A recorded IK first message can be replayed, and only the responder can answer it, so its recorder can test whether the same responder is still present, though it learns no key. A live session ends when a new handshake with the peer completes, or by silence (Section 2.5). The Curve25519 static key carried by Noise is the birational image of the agent's Ed25519 identity key, so the expected static is computed from the peer's Ed25519 public key. The handshake is a strict request–response exchange: each message is sent only after the previous one arrived; over a lossy carrier each handshake message is retransmitted on a timer until the next one arrives, and the final one, which has no reply (message 2 of IK, message 3 of XX), until the other side's attestation message arrives (below), transport messages are sent only after the handshake completes, and the session's replay protection makes duplicated packets harmless. Sessions are per underlay: one handshake per underlay per pair, so a pair reachable over more than one underlay holds a session on each. A peer becomes reachable over an underlay (Section 2.3) once its session on that underlay is established and authenticated, and the attestation exchange on it has completed, with the peer either attested or declared unattested.

Attestation. The handshake is followed by a mutual attestation exchange, and neither peer is reported reachable until it completes. The exchange has two parts, one long-lived and one per session. Once, the layer asks the platform to generate an *attestation key* in its secure element and to attest it over a challenge naming the agent's identity public key pk: App Attest on iOS, hardware key attestation on Android. For each session, the layer asks the platform to sign with that key the digest $H(\text{“glp attest”} \mid pk \mid v \mid h)$, where v is the application's version number and h is the final Noise handshake hash. Each side sends one attestation message per session, as a message of the Message Protocol (Section 2.10) after the final handshake message: either its attestation and this signature, or a statement that its platform provides

no attestation. The receiver verifies the attestation against the platform's root, and the signature against the key the attestation carries; either failing tears the session down, while the identity of a peer that sends the statement is reported unattested: `onPeerConnected` carries `null` for the application. Since any agent can declare itself unattested, including one running a repackaged application or holding a copied identity key, attestation protects only the uses GLP restricts to attested peers. The receiver waits for the peer's attestation message as long as the sender's attempts to deliver it last, then tears the session down. A payload arriving on a session whose exchange has not completed is dropped unacknowledged, so the sender's retransmission delivers it once the exchange has completed.

The attestation binds the attestation key to the application and to the identity key, and the signature binds both, with the version number, to this session. We bind the signature to the handshake hash because an identity key is long-lived: an attestation over the key alone could be replayed onto every session the agent opens, while h is unique per session, identical at both ends, and chosen by both sides. The attestation itself is long-lived, since a platform fixes the challenge when it generates the key, and a per-session challenge would need a new secure-element key per session. On iOS the attestation key remains valid through updates of the application, so an updated application needs no new attestation and signs its new version number in each new session; reinstalling the application, migrating it to another device or restoring the device from a backup needs a new attestation key [1]. A peer that restarts proves itself again before it is reachable. The operating system above the runtime relies on two properties of this exchange: an attestation cannot be replayed onto another session, and the layer only verifies attestations, while GLP decides what an unattested peer, such as a rendezvous server (Section 6), may be used for.

The key the platform holds is the attestation key, not the identity key. The identity key is Ed25519 with the Curve25519 static above, and hardware key stores hold other curves—Apple's Secure Enclave holds P-256 only—so the identity key lives in the application's own store. The attestation key is generated in the secure element and stays in it; the attestation binds the identity key to it once, and the signature binds it to each session. A copy of an identity key therefore opens only unattested sessions on a machine that does not run the attested application, and a genuine application that imports keys, for example to move an identity to a new smartphone [2], attests it like any key it generated. Attestation proves that a genuine application holds the key.

The two platforms prove different things, and the difference bounds the claim above. Both attestations are verified by the peer against a public root—Apple's App Attest root, Google's hardware attestation root—so no third party is contacted at the moment two agents establish a session, which is the property the protocol needs. Google's Play Integrity is not used: its verdict is opened only by Google's servers, so proving oneself to a peer would require reaching a global service on every session. What Android's certificate chain establishes is that the attestation key lives in the device's secure hardware and that the application holding it carries the developer's signing certificate the attestation names; on a device whose operating system is modified that certificate can be forged. The claim above therefore holds against a repackaged application on both platforms, and against a modified operating system on neither.

Only the device's hardware root can prove which application the device runs, and the platform's vendor holds that root; neither code the application ships nor a server can prove

it, since each sees only what the device sends. What the platforms do require of a third party lies outside the session, and the two differ. On Android the attestation certificates are provisioned to the device by Google and, from Android 16, only so, and revocation is published by Google as a status list a verifier fetches; so a device that has never reached Google cannot attest, and a verifier that never reaches it does not learn that a key has been revoked. On iOS the corresponding contact is on the producing device alone: attesting a key reaches an Apple server once, and Apple documents no revocation, so an iOS verifier contacts nobody. Neither platform is contacted during a session, and an agent that cannot attest is reported unattested (below) rather than refused, so a session between two agents remains a matter for the two of them.

What the attestation names is the application published under its developer's key. To report the same on both platforms, we take the attested application to be its developer, its name and its version number. Android's certificate names the digest of the developer's signing certificate, the package and its version; App Attest names the application identifier the key was attested under, the developer's team identifier and the bundle identifier, and no version. On both platforms the version number is the one the application signs in the per-session digest, which only the genuine application can sign. Neither platform measures the running code, so that one build runs under one name and version number is the developer's undertaking, which the operating system above assumes.

A headless server profile (Section 2) has no attestation to offer: it sends the statement that its platform provides no attestation, and its peers report it unattested; whether to transact with an unattested peer is a GLP-level decision. An attestation offered and found invalid tears the session down.

2.7 Geofencing

The location service is the platform's geofencing, and the layer uses it for one purpose: to tell GLP when the smartphone enters or leaves a place GLP has declared. A place is a geofence, a circle of a given radius in metres about the location the device is at when the place is declared, registered with the platform under a name GLP chooses. With `declarePlace(place, radius)` the layer registers the geofence, and a second declaration of the same place replaces the first; with `removePlace(place)` it unregisters it (Section 5). The platform then watches the device's position and reports each crossing of a registered geofence to the layer, which passes it on through `onPlaceEvent(place, event)` as `entered` or `exited`. The event carries the name of the place and the direction of the crossing. What a place means, for example that the person declared the place they are in trusted, so PAN contact there is open (Section 2.8), is decided by GLP.

The platform does not always report crossings: it stops when the application moves to the background or when the person withdraws the location permission, and resumes when either is reversed. Without knowing this, GLP could not tell a device that stayed inside a place from one whose exit went unreported, so the layer reports `unobservable` when the platform stops reporting and `observable` when it resumes. Observability belongs to the platform's reporting and not to one geofence, so both events are delivered for every standing place at once, and a place declared while reporting is stopped is told so as soon as it is declared. A crossing that occurs while the device is unobservable is lost. On iOS, the platform reports crossings to an

application in the background only under the permission to use location at all times, which we do not request, so the platform supplies the position only in the foreground: the layer decides the crossing itself, and reports the device unobservable whenever the application is not in the foreground. A headless server has no location service: its layer is built without one, and refuses `declarePlace` and `removePlace`.

2.8 Cold Calls

A *cold call* is a first-contact connection attempt between two agents that do not already share an active communication channel. A cold call is realized in one of three ways: by an encounter on the PAN (Section 3.3.2), by a LAN encounter (Section 3.2), or over the WAN between agents whose programs have declared each other's keys (Section 4.4). What a cold call *means* is a GLP-level interpretation and is not part of the networking layer.

Trust levels. Whether the networking layer answers an *unsolicited* proximity encounter from an unknown key (Section 3.1) is governed by a trust level set by GLP and enforced by the layer:

- **Open:** The agent completes the ANNOUNCE exchange and the Noise XX handshake with any unknown contact (Section 2.6), thereby revealing its full public key and allowing first contact.
- **Closed:** The agent answers only an IK handshake from a known key, so it reveals its key only to peers that already hold it.

A level is held per proximity underlay, and is set with `setTrustLevel(level)` on that underlay's sub-object, `pan` or `lan` (Section 2.2); both levels default to Closed. GLP sets the two levels independently, for example opening PAN contact while the smartphone is inside a place the person declared trusted (Section 2.7), and LAN contact while it is attached to a local network the person declared trusted (Section 4.3).

Unsolicited inbound WAN contact from outside the agent's own local network is answered only from a peer in the known-peer set (Section 2.4), by the IK handshake (Section 2.6). A rendezvous server, marked so when its layer is instantiated (Section 2.2), answers unsolicited WAN contact from any key, since it cannot know its clients in advance (Section 6).

LAN contact arrives on the same path, LAN and WAN connections being both IP connections, so the layer tells the two apart by the source address: an unsolicited inbound contact whose source address lies within one of the agent's own local address prefixes is LAN contact, governed by the LAN level.

2.9 Point-to-Point Communication

The networking layer provides reliable delivery of arbitrary byte payloads between peers, identified by public key.

Function	Signature	Description
Send payload	`send(pubKey, payload)`	Send a byte payload to the peer identified by `pubKey`; the call completes once the message is saved for delivery (Section 2.11), and fails if the message cannot be saved.
Receive payload	`onMessageReceived(sender, payload, messageId, underlay)`	Register a callback for incoming payloads. Receives the sender's public key, the byte payload, a layer-assigned `messageId`, and the `Underlay` over which the message was delivered.
Acknowledge delivery	`acknowledge(messageId)`	Confirm that the message delivered under `messageId` has been applied and saved by the runtime; the layer keeps a delivered message until it is acknowledged, and delivers it again at restart otherwise (Section 2.11).

2.10 Message Protocol

One message protocol carries `send` payloads, and the attestation exchange of Section 2.6, on every underlay. Every packet bears the five-byte frame: a packet-type byte and a 4-byte big-endian payload length. A message is assigned its `messageId` and split into fragments sized to the carrier, each fragment carrying the `messageId`, its index, and the fragment count, encrypted under the session of Section 2.6. The receiver acknowledges fragments; the sender retransmits unacknowledged fragments with backoff, under a bounded send window per peer. When all fragments have arrived the message is reassembled and delivered whole—atomic delivery (Section 2.11). A re-send after a lost acknowledgment, possibly over another underlay, may deliver a message multiple times, and madGLP dedupes repeated deliveries [8]. When a message's attempts are exhausted the session is torn down and the message queues for the next path (Section 2.5). Messages are unordered (Section 2.11); a lost fragment delays only its own message.

Over LAN and WAN a fragment rides a UDP datagram, sized within the path MTU; over BLE it rides a GATT write without response, sized to the negotiated MTU; on every

underlay retransmission is the reliability mechanism.

2.11 Networking Assumptions

The networking layer must satisfy the following assumptions for madGLP correctness:
Fair message delivery: Every message passed to `send()` is eventually delivered to the recipient at least once, assuming the recipient eventually becomes reachable. Messages to temporarily-unreachable peers are queued and delivered when connectivity is restored; the queue survives the termination of the application. A message is delivered until the runtime acknowledges it with `acknowledge(messageId)` (Section 2.9), which it does once the state that applied the message is saved [8]: the receiving layer keeps a delivered message until it is acknowledged, across the termination of the application, and delivers it again when the application restarts.
Unordered delivery: Messages between two agents may be delivered in any order, and the madGLP runtime tolerates any order. The networking layer is free to route concurrent messages to the same peer over different underlays.
Atomic message delivery: Each message is delivered as a complete unit. If a message is fragmented across multiple packets, the networking layer reassembles them before delivery.
Integrity: Messages are not corrupted in transit. The networking layer provides encryption and integrity checks.
Authentication: The sender's identity (public key) is authenticated. A received message from public key pk was indeed sent by the agent holding the corresponding private key.

3 Proximity Underlays: PAN and LAN

An agent discovers and contacts agents in its vicinity—on the same local network, or within radio range—without any infrastructure and without becoming trackable. This section specifies the shared design, then its two realizations: PAN and LAN.

3.1 Concepts and Principles

Recognition before contact. Each proximity underlay gives an agent a public, passively observable presence—a BLE advertisement, an mDNS service instance. That presence must let a peer who already knows the agent recognize it before any contact, yet give everyone else nothing stable to track. Both realizations therefore advertise the same construction: a rotating 8-byte *recognition token*, the first 8 bytes of $H(L \mid pk \mid T)$, where H is SHA-256, pk the advertiser's public key, T the current 15-minute time slot, and L a per-underlay label. Anyone who knows pk computes the token for the current and adjacent slots and matches it against observed presences; adjacent-slot matching absorbs clock skew. To anyone else the token is an identifier that changes every slot. The advertisement carries the token.

Contact and identity. A recognized presence is contacted on its underlay with the IK handshake of Section 2.6, whose expected key is the recognized peer's. Under Open trust any

other Grassroots presence is contacted too: the peers exchange ANNOUNCE (Section 2.5), binding the contact to a claimed public key, and run the XX handshake, which checks the claim. Where either endpoint may initiate the connection, the deterministic initiator is the endpoint with the lexicographically smaller public key, as everywhere (Section 4.1). Addresses and contacts observed by proximity discovery feed the layer's connection attempts directly; GLP distributes addresses (Section 4.2).

Cold-call trust levels. The trust levels of Section 2.8 govern proximity encounters as unsolicited contact, one level per proximity underlay: under Open the agent contacts or answers unmatched presences on that underlay, revealing its full public key in ANNOUNCE; under Closed the advertised token serves only to recognize already-known peers (Section 2.4) and unmatched encounters are ignored. The PAN and LAN underlays carry independent trust levels, so an agent may answer strangers over one underlay while ignoring them over the other.

Discovery. Proximity discovery is exposed to the runtime by the same functions on both proximity sub-objects, `pan` and `lan` (Section 2.2), each acting on its own underlay:

Function	Signature	Description
Scan for peers	`getPeers()` → `List<Peer>`	Look for nearby Grassroots agents on this underlay: on `pan` a BLE scan for the Grassroots prefix, on `lan` an mDNS browse for the service type.
Peer discovered	`onPeerDiscovered(pk)`	Callback for each nearby Grassroots agent found on this underlay.
Set trust level	`setTrustLevel(level)`	Set the cold-call trust level of this underlay to Open or Closed (Section 2.8).

3.2 LAN

Two agents on the same local network can find each other and communicate directly. Discovery uses DNS-SD over multicast DNS: each agent advertises an instance of the service type `_grassroots._udp` on the local domain and browses for instances of it. The advertised instance name is the recognition token of Section 3.1, hex-encoded, with label `"glp lan suffix"`; the instance's SRV record carries the agent's LAN address and bound UDP port.

A matched instance yields an address, and contact proceeds as over the WAN: a UDP path with the deterministic initiator (Section 4.1) and the session of Section 2.6, preceded by ANNOUNCE only for an unrecognized instance under Open trust; reachability reports the underlay `LAN`, and communication is that of Section 4.5.

3.3 PAN

Bluetooth Low Energy enables offline, infrastructure-free operation between devices in direct radio range of each other, and is the bootstrap underlay where agents first meet face-to-face.

3.3.1 Advertising and Recognition

Each agent advertises a BLE GATT service whose 128-bit (16-byte) UUID has two parts: a fixed 8-byte *Grassroots prefix* common to all Grassroots agents, enabling general discovery of any nearby Grassroots agent, and the rotating 8-byte recognition token of Section 3.1 with label `"glp ble suffix"`. At each slot boundary the token is recomputed and advertising is restarted with it. When an app is not in the foreground, iOS advertises the service only to a scanner for that exact UUID, so a friend who computes the token still finds the agent; discovery by strangers needs the app in the foreground.

3.3.2 Connection and Identity

Upon BLE connection to a recognized presence, the peers establish the IK session of Section 2.6 over the GATT connection directly; to an unrecognized presence under Open trust, they first exchange ANNOUNCE, which carries the agent's full 32-byte public key, and then run the XX handshake. Where either endpoint may initiate—both run central and peripheral—the deterministic initiator is the endpoint with the lexicographically smaller public key, as over the WAN (Section 4.1).

3.3.3 Communication

`send(pubKey, payload)` transmits the byte payload over the established GATT connection via the message protocol of Section 2.10, GATT writes as the carrier; fragmentation and reassembly satisfy atomic delivery (Section 2.11). `onMessageReceived` delivers reassembled payloads to the runtime.

4 WAN Underlay

This section specifies how the networking API (Section 2) is realized over the WAN, the Internet at large. Communication uses datagrams carrying the message protocol of Section 2.10 over UDP sockets. NAT traversal is coordinated by a rendezvous agent, on a server or on a well-connected friend's smartphone (Section 6). The rendezvous protocol is GLP code (Section 6).

4.1 WAN Connection

Once a UDP path is established (directly or by hole-punching), the peers establish the session of Section 2.6 over it. When both endpoints might initiate—mutual LAN discovery,

simultaneous reconnection—the deterministic initiator is the endpoint with the lexicographically smaller public key. UDP has no connections, so ANNOUNCE is the liveness check (Section 2.5): a peer silent for two intervals is disconnected.

4.2 Connectivity and Address

Mobile agents typically run behind NAT without static IP addresses, and an agent cannot directly observe its own public address. When its address changes (e.g., due to roaming), all its IP-based paths break. The networking layer reports address changes to the runtime, and exposes the agent's current public address once known (an agent learns it from a rendezvous server, which observes the source address of the agent's packets; see Section 6). GLP code uses these to share the agent's address with its peers and to trigger reconnection. The address is GLP data and travels over any underlay: GLP can hand a fresh address to a BLE-adjacent friend, which can relay it to remote friends and coordinate their reconnection via the system predicates (Section 5), serving as an edge rendezvous point without a globally-routable address.

Address-change recovery. When an agent's own address changes, its established UDP paths break and their sessions are torn down by liveness (Section 2.5), but its ability to initiate outbound connections is unimpaired: the recorded addresses of its known peers did not change. The heartbeat sweep therefore re-initiates connections to the known peers and the declared rendezvous servers; the rendezvous connection always succeeds (globally-routable target), restoring the RV channel and, through it, the agent's fresh public address. Recovery is thus asymmetric by design: the endpoint that moved reconnects; a stationary peer is reconnected to. Distributing the fresh address to peers that cannot be reached directly (both behind NAT) is GLP's: GLP learns the address from `onConnectivityStatusChanged`/`getPublicAddress`, sends it to friends over whatever channels remain live—rendezvous-mediated signaling, a BLE-adjacent friend, surviving sessions—and it enters each peer's layer via `putPeerAddress`, whereupon that peer's own sweep connects to it, punching if coordinated (Section 5).

Rendezvous servers. The rendezvous servers the layer connects to are those GLP has declared, each by its public key and its address, and GLP adds and removes them while the layer runs. GLP knows which of its peers are rendezvous servers by the keys it declared.

The functions below are called on the layer's `wan` sub-object (Section 2.2).

Function	Signature	Description
Connectivity changed	`onConnectivityStatusChanged(oldAddress?, newAddress?)`	Callback when the networking layer detects a change in the agent's public IP address. Receives `(oldAddress?, newAddress?)` as `ip:port` strings, either of which may be `null` when no public address is available (e.g. startup: `old=null`; loss: `new=null`; update: both present and different). Fired on startup and on every address change.
Public address	`getPublicAddress()` → `Address?`	The agent's current public `ip:port`, or `null` if not yet known.
Feed peer address	`putPeerAddress(pk, address)`	Inform the networking layer of a peer's current public `ip:port`, learned at the GLP level; the layer records it with the known peer (Section 2.4) for `send` and reconnection. One WAN address is recorded per peer; the latest, whether GLP supplied it or the layer observed it, replaces the earlier.
Observed address	`observedPeerAddress(pk)` → `Address?`	The public `ip:port` at which the layer observes the connected peer, or `null` if none is observed; behind `peer_address/2`, which reports `address(S)` or `none` [6].
Punch	`punchUdp(address)`	Send toward the address a short burst of UDP datagrams to open the NAT mapping; behind `punch_udp/1`.

Function	Signature	Description
Declare server	`putRendezvousServer(pk, address)`	Add the rendezvous server with public key `pk` at `address` to the servers the layer connects to; a further declaration of the same `pk` replaces its address; behind `declare(rendezvous(Key, Address))`.
Remove server	`removeRendezvousServer(pk)`	Remove the server from those the layer connects to, and end the layer's session with it; a key not declared does nothing; behind `remove(rendezvous(Key))`.

4.3 Local Network Identity

The `wan` sub-object (Section 2.2) reports an opaque fingerprint of the local network the agent is attached to, and notifies the runtime when it changes. The fingerprint is derived from whichever of these the platform provides without additional permission: the local address prefix, the default gateway address, and the wireless network name.

This is not `onConnectivityStatusChanged` (Section 4.2). The two are independent in both directions: the attached local network can change while the public address does not, and the public address can change while the attached local network does not.

The fingerprint is best-effort: two distinct local networks may yield the same one, and a platform may make none of its constituents readable. A match is evidence that the agent is attached to the same local network as before, not proof of it; an agent compares a fingerprint only against its own earlier values.

Function	Signature	Description
Network identity	`networkIdentity() → NetworkId?`	An opaque fingerprint of the local network the agent is currently attached to, or `null` when it is attached to none or no constituent is readable.
Network changed	`onNetworkChanged(oldId?, newId?`	Callback when the fingerprint changes. Receives `(oldId?, newId?)`, either of which may be `null`.

4.4 WAN Cold-Call

Over the WAN, a cold call is a connection between two agents whose programs have each declared the other's key to their layer (Section 2.4). How the two learn each other's keys, by a signed one-time invitation link redeemed at a rendezvous server, is Section 6. To the layer each is a known peer of the other: the two connect over UDP, punching where needed (Section 4.2), and establish the session of Section 2.6; only then does the caller send the madGLP cold-call payload to the callee's index-0 serializer.

4.5 WAN Communication

`send(pubKey, payload)` transmits the byte payload to the peer as session-encrypted fragments in UDP datagrams, per the message protocol of Section 2.10. `onMessageReceived` delivers complete, reassembled messages to the runtime.

5 The Seam: Layer Functions behind the System Predicates

The GLP runtime realizes five system predicates over the networking-layer functions this section specifies. These predicates give GLP agents the abilities of address observation, reading the application a peer attested, UDP hole punching, declaration of places as geofences, trust level setting, known-peer set management including agent addresses, and declaration of rendezvous servers. The predicates are `peer_address/2`, `peer_app/2`, `punch_udp/1`, `declare/1` and `remove/1`; a declaration is one of `peer(Key)`, `address(Key, Address)`, `rendezvous(Key, Address)`, `trust(Underlay, Level)` with the underlays `pan` and `lan` and the levels `open` and `closed`, and `place(Name, Radius, Events)`, and a removal is one of `peer(Key)`, `rendezvous(Key)` and `place(Name)`. They are defined in the language paper's catalogue [6], and their clauses in the implementation paper [8]; only the functions behind them are specified here.

If an underlay changes—for example, UDP datagrams replaced by QUIC, or hole-punching by another traversal mechanism—only the layer's implementation of these functions changes; the GLP code that uses the predicates is unaffected.

The layer functions. The functions behind the predicates are specified with their underlay or subject: `observedPeerAddress` and `punchUdp` with the WAN, the attested application of `onPeerConnected` behind `peer_app/2` with the sessions (Section 2.6), on the `wan` sub-object (Section 4.2), `setTrustLevel` with the trust levels, on the `pan` and `lan` sub-objects (Section 2.8), `putKnownPeer` and `removeKnownPeer` with the known peers, on the layer (Section 2.4), and `putPeerAddress`, `putRendezvousServer` and `removeRendezvousServer` with the WAN, on the `wan` sub-object (Section 4.2). The place functions are specified here:

Function	Signature	Description
Declare place	`declarePlace(place, radius)`	Register with the platform a geofence of `radius` metres about the device's current location, named `place`; a further declaration of the same `place` replaces the first; behind `declare(place(Name, Radius, Events))`.
Remove place	`removePlace(place)`	Unregister the geofence of `place`; a `place` not declared does nothing; behind `remove(place(Name))`.
Place events	`onPlaceEvent(place, event)`	Callback with the `place` and an `event`, one of `entered` and `exited`, as the platform reports the device crossing the geofence, `unobservable` when the platform stops reporting—the application in the background, a permission withdrawn—and `observable` when it resumes; feeds the stream of `declare(place(Name, Radius, Events))`.

Signatures. The layer provides `sign(bytes)` and `verify(pk, bytes, signature)`, Ed25519 signing and verification under the agent's key and against a public key. The runtime realizes the language's `sign/3` and `signature/2` over them [6], adding the source identity of the signing program.

6 Rendezvous Server

A *rendezvous server* is a separate process—a host that provides a static, globally-routable IP address and runs a lightweight GLP *rendezvous agent*. Its purpose is to give NAT-bound agents, which cannot otherwise reach one another, a fixed point they can always reach outbound, from which they discover their own public addresses and coordinate UDP hole-punching.

This section specifies the rendezvous server whole: the networking requirements, what a machine must provide to host one, and the rendezvous agent, the GLP program that runs on it, with the two protocols the social graph above the runtime runs through it, reconnection

and the redemption of an invitation. The social graph's side of each, when it asks and what it does with the answer, is the operating system's (the Grassroots Super-App, in preparation).

Networking requirements.

- **Static, globally-routable address.** The host has a fixed public `ip:port` that any agent can reach outbound, traversing NAT in the outbound direction without assistance.
- **Address observation.** When an agent connects, the rendezvous server observes the source `ip:port` of the agent's packets and exposes it via `observedPeerAddress` (Section 4.2). This is how a NAT-bound agent learns its own public address.
- **Hole-punch coordination.** The rendezvous server relays observed addresses between two agents, and each sends punch traffic toward the other with `punchUdp` (Section 4.2) so a direct UDP path can form. Once the path is open the peers run the Noise handshake and the message protocol directly (Sections 4.1 and 2.6); the rendezvous server sees signaling metadata and observed addresses.
- **No relay.** The rendezvous server forwards no madGLP payload. When a direct path cannot be established (e.g., symmetric NAT), a connected common peer relays, in GLP.
- **Any key.** A rendezvous server cannot know its clients in advance, so its layer, marked so when it is instantiated (Section 2.2), answers unsolicited WAN contact from any key (Section 2.8); it belongs to no social graph.
- **Unattested.** Having no platform attestation to offer (Section 2.6), a rendezvous server is reported unattested, and this costs nothing: a false address fails to connect, and it cannot make an unattested peer appear attested, attestation being verified between the two agents themselves. GLP knows a rendezvous server by the key it declared (Section 4.2). What an unattested channel may then be used for is a GLP-level decision.

Any number of servers. Any individual or organization can run a rendezvous server; the servers do not communicate with one another. An agent may use several, which GLP declares (Section 4.2); the system operates with none when agents are on the same PAN or LAN (Section 3). In the terminology of the ICE framework [4], a rendezvous server combines the roles of STUN server (address discovery via `observedPeerAddress`) and application-specific signaling server (candidate exchange and hole-punch coordination, performed by the rendezvous agent).

The contract. What a rendezvous server does is the grassroots contract GRV, which extends GSG, the social graph [3], and from which the rendezvous agent is compiled; it runs on the server under its owner's key, with no person and no screen. The text in the box is the repository's copy, `programs/examples/rendezvous/GRV.txt`.

GRV: Grassroots Rendezvous

GRV extends GSG [3].
Alice, Bob and Carol are persons. A *rendezvous server* is a machine of Carol's at an address every person can reach. An *invitation link* is a term Alice signs, naming Alice, a rendezvous server, a serial and an expiry.

1. ■ *Server.* Carol may set up a rendezvous server.
2. ■ *Registration.* Alice may register with Carol's server an invitation link she signed that names the server, with a serial she has not registered with it before.
3. ■ *Redemption.* Bob may present to Carol's server an invitation link registered with it that has not expired; the server then gives Alice Bob's address and Bob Alice's, and the link is spent.
4. ■ *Reconnection.* Alice and Bob, who are friends, may each present to Carol's server the other's signature over their friendship; the server then gives each the address of the other.

Implementation pragmas

Redemption, Reconnection: the address of a person is the one at which the server's networking layer observes them, read by `peer_address/2`; a person given an address punches towards it by `punch_udp/1`, and the smaller key initiates.
Registration: an invitation link expires an hour after it is made.

The rendezvous agent. The rendezvous agent is a GLP program, `programs/examples/rendezvous` in the GLP repository, run on the headless layer under the server's key; it holds no social graph, takes nothing when it starts, keeps no record beyond the requests awaiting their match and the serials of the links it has matched until they expire, and acts through the seam predicates: `peer_address/2` for the address at which the layer observes a requester, and the answer it sends, on which the requester calls `punch_udp/1`. A request reaches it as a cold call over the WAN from any key (above), and it answers a request only where `signature/2` reports the signed term it carries signed by the key the request names.

Reconnection. When two friends become friends, each signs with `sign/3` the one pair `attest(PkA, PkB)`, `PkA` the lexicographically smaller key, and gives the signature to the other, so each holds the other's signature over their friendship and neither can forge the one it presents. An agent that has lost the path to a friend sends the rendezvous agent, over the session it can always open, a reconnection request naming the friend and carrying the friend's signature; the friend, having lost the path, does likewise with the requester's. The rendezvous agent matches the pair, verifies each signature as the other party's, reads the address at which the layer observes each by `peer_address/2`, refusing where it observes none, and relays to each the address of the other; each calls `punch_udp/1` towards it, and the deterministic initiator connects (Section 4.1). The request, the match and the relay are

GLP messages of the rendezvous agent's program; the layer supplies the observed addresses and the punch.

Invitation links. An agent that invites a person it has no path to generates an *invitation link*: the term `invitation(I, Rv, RvAddr, Serial, Expiry)`, the inviter's key, a rendezvous server's key and address, a serial the inviter keeps, a whole number used once, and an expiry, an hour after creation in the unit of `now/1`, signed by the inviter with `sign/3`; and registers it, over its session with that server, by `register(I, Link, Reply)`, which the rendezvous agent admits where the link is signed by `I` and `I` has not registered that serial before, used or not, and rejects otherwise; the inviter's person sends the link, as text, over any channel they have. The recipient's agent verifies the inviter's signature and the expiry, declares the inviter a peer, and redeems the link at the rendezvous agent named in it by `redeem(R, Signed, Reply)`, `Signed` the term `redeem(R, Link)` signed with its own key, a signed term being ground. The rendezvous agent accepts the redemption only where the link names it, is signed by the inviter it names, has not expired, `now` being below `Expiry`, the redemption is signed by `R`, and the serial is registered for the inviter and unused; it then marks the serial used, so that a link admits one first contact, reads the address at which the layer observes each of the two by `peer_address/2`, rejecting where it observes none and changing nothing, and answers each with `punch(Address)`, the address of the other; each punches, the smaller key initiates, the two establish their session, and the recipient's first message is a cold call (Section 4.4). An unredeemed registration waits until its serial is redeemed; a used serial is kept, with the registered link's expiry, until that expiry and then dropped, after which the inviter may register the number again, a serial being used once by the inviter's own keeping. What the two then do, befriending or not, is the social graph's.

Versions. The rendezvous agent admits a request by its keys: the request is signed by the key it names, and the two signatures of a pair are the two friends' over their pair. So an update of the application, which changes the source identity a signed term carries (Section 5), leaves the signatures friends hold good. The application has a version, which its attestation carries (Section 2.6), and an earliest version it is compatible with; on a friend channel's handshake the two social graphs exchange both and go on only if the two ranges intersect, above this layer (the Grassroots Super-App, in preparation).

7 Complete API Summary

The following table summarizes the complete networking API. The first five groups are underlay-agnostic and called on the layer; the Proximity group is called on the `pan` and `lan` sub-objects, and the WAN group on the `wan` sub-object (Section 2.2); the system predicates are the language's [6], and the layer functions behind them are in Section 5.

Category	Function / Predicate	Direction	madGLP Role
Identity			

Category	Function / Predicate	Direction	madGLP Role
Query	`getIdentity()`	Runtime ← Network	Retrieve agent public key
Known peers			
Add	`putKnownPeer(pk)`	Runtime → Network	Supply a key the layer recognizes
Remove	`removeKnownPeer(pk)`	Runtime → Network	Withdraw recognition and recorded address
Connection & Reachability			
Connect	`onPeerConnected(pk, underlay, application?)`	Runtime ← Network	Peer reachable on an underlay; pk, underlay, and the attested application, or null for an unattested identity
Disconnect	`onPeerDisconnected(pk, underlay)`	Runtime ← Network	Peer unreachable on an underlay; queue messages
Reachable	`isPeerReachable(pk)`	Runtime → Network	Check before send
Transports	`peerUnderlays(pk)`	Runtime → Network	Underlays now reaching pk (proximity)
Communication			
Send	`send(pubKey, payload)`	Runtime → Network	Implements madGLP Send transaction
Receive	`onMessageReceived(sender payload, messageId, underlay)`	Runtime ← Network	Feeds madGLP Receive transaction
Acknowledge	`acknowledge(messageId)`	Runtime → Network	The message was applied and saved; the layer may forget it
Cryptography			
Sign	`sign(bytes)`	Runtime → Network	Ed25519 signature under the agent's key

Category	Function / Predicate	Direction	madGLP Role
Verify	`verify(pk, bytes, signature)`	Runtime → Network	Check an Ed25519 signature by a public key
Proximity underlays (PAN and LAN)			
Scan	`{pan,lan}.getPeers()`	Runtime → Network	Find nearby agents on that underlay
Notify	`{pan,lan}.onPeerDiscovered(pk)`	Runtime ← Network	Nearby Grassroots agent on that underlay
Policy	`{pan,lan}.setTrustLevel(`	Runtime → Network	Set that underlay's cold-call trust level
WAN			
Connectivity	`wan.onConnectivityStatus new?)`	Runtime ← Network	Public IP change; (old?, new?)
Own address	`wan.getPublicAddress()`	Runtime ← Network	Current public ip:port, share with peers
Local network	`wan.networkIdentity()`	Runtime ← Network	Fingerprint of the attached local network
Network change	`wan.onNetworkChanged(oldId?, newId?)`	Runtime ← Network	Local network changed; (old?, new?)
Feed address	`wan.putPeerAddress(pk, address)`	Runtime → Network	Hand the layer a GLP-learned peer address
Observed address	`wan.observedPeerAddress(pk)`	Runtime → Network	Public address at which a connected peer is observed
Punch	`wan.punchUdp(address)`	Runtime → Network	UDP hole-punch toward an address
Declare server	`wan.putRendezvousServer(pk, address)`	Runtime → Network	A rendezvous server the layer connects to
Remove server	`wan.removeRendezvousServ`	Runtime → Network	Withdraw it
Places			
Declare place	`declarePlace(place, radius)`	Runtime → Network	Register a geofence about the device's location

Category	Function / Predicate	Direction	madGLP Role
Remove place	`removePlace(place)`	Runtime → Network	Unregister it
Place events	`onPlaceEvent(place, event)`	Runtime ← Network	Crossings, and when they cannot be observed

References

[1] Apple. Establishing your app's integrity. Apple Developer Documentation, `https://developer.apple.com/documentation/devicecheck/establishing-your-app-s-integrity`, 2026. Accessed 22 October 2026.

[2] Ohad Eitan, Idit Keidar, and Ehud Shapiro. Securing people and their machines against major faults (full version). *Proceedings of The 28th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS 2026), to appear*, 2026. arXiv:2607.02304.

[3] James Golike, Andy Lewis-Pye, and Ehud Shapiro. Formalising and implementing grassroots social contracts: From legal text to grassroots platforms (full version). *arXiv:2609.06231*, 2026.

[4] Ari Keranen, Christer Holmberg, and Jonathan Rosenberg. Interactive connectivity establishment (ICE): A protocol for network address translator (NAT) traversal. RFC 8445, 2018.

[5] Ehud Shapiro. Grassroots distributed systems: Concept, examples, implementation and applications (brief announcement). In *37th International Symposium on Distributed Computing (DISC 2023). (Extended version: arXiv:2301.04391)*, pages 47:1–47:7, Italy, 2023. LIPICS.

[6] Ehud Shapiro. Glp: A grassroots, multiagent, concurrent, logic programming language for ai (full version). *arXiv preprint arXiv:2510.15747, Summary in Proc. of ICLP'26, EPTCS 450, pp. 119–133*, 2025.

[7] Ehud Shapiro. Grassroots platforms with atomic transactions: Social graphs, cryptocurrencies, and democratic federations. In *Proceedings of the 27th International Conference on Distributed Computing and Networking*, pages 71–81, 2026. arXiv preprint arXiv:2502.11299.

[8] Ehud Shapiro. Implementing grassroots logic programs with multiagent transition systems and ai (full version). *arXiv:2602.06934. Summary to appear in Proc. of LOPSTR+PPDP'26*, 2026.

A Implementation Notes

This appendix records the implementation of the networking layer against the specification above: where the code is and how it is organised, the rules it is written by, what of the specification it realizes, what it decides below the specification's level of detail, and what deviates or is not yet implemented. A rule of this appendix that the specification contradicts is a defect in one of the two, to be reported and not resolved silently.

A.1 The Code

The layer is `networking` in the public GLP repository, imported as a subtree from the `grassroots-networking` repository of Dan Bachar; the paper is its specification. It is written in Dart. The protocol internals are the pure-Dart package `networking/core`: the wire frame, ANNOUNCE, the Noise sessions, the attestation exchange, the message protocol, the UDP transport, the known-peer set, the hole-punch service and the headless profile. The Flutter package, `networking`, depends on the core and re-exports it, adding BLE in both roles, DNS-SD, the platform's attestation and geofencing bindings and the demo application; it is one source, not a fork of the core. The glue by which the GLP runtime reaches the layer, `HeadlessGlpNetwork` and `GrassrootsGlpNetwork`, is the package `networking/glue`, which depends on the core, the Flutter package and `glp_runtime` by path, so the core and its users carry no GLP runtime, and a fork of the layer is a fork of the whole repository.

The code's names are the paper's: `Underlay` with the values `pan`, `lan` and `wan`, `ProximityUnderlay` with `pan` and `lan`, and the sub-objects `pan`, `lan` and `wan` of Section 2.2, each `null` for an underlay the layer does not run. The layer is constructed with the parameters of Section 2.2. The constructor also takes the platform's two services, the attestation binding behind `PlatformAttestation` and the geofencing binding behind `PlaceGeofenceBackend`, which each build's entry point passes for the platform it is built for; this is how a build fixes the two services the specification has fixed by the platform (Sections 2.6 and 2.7). The headless profile takes no underlays, running the WAN, no location service, and optionally a listen port and a static public address, as one pair; it has the `wan` sub-object, refuses `declarePlace` and `removePlace`, and has no trust level, the known-peer set being the whole of what it answers. A rendezvous server is the headless profile marked as one, with the GLP rendezvous agent over it through the glue, built from `glue/bin/rendezvous_server.dart` with `dart build cli`, since `dart compile exe` refuses the glue package's Flutter dependency. The headless embedders, `glp_net_push`, `glp_net_service` and the rendezvous server, each name the file of the layer's queue, `-outbound-queue-file`, a required argument. The simulation realization of the interface lives in the GLP runtime and is recorded in the implementation paper's Implementation Notes [8].

A.2 Rules of the Code

No legacy or compatibility code. A change replaces what it changes: no wrapper kept for compatibility, no dead code, every call site updated. This holds for the wire format too: a field added to a packet is required, a payload without it is malformed and refused, and no decoder tolerates a form of the packet that no longer exists. There is no installed application

in the wild, so the wire format changes whenever the design improves; the protocol version carried in ANNOUNCE and the handshake envelope rises with each incompatible change, and a peer on an earlier version is refused.

Local queueing, no relaying. A message goes from its sender to its recipient directly. The sender's own messages are queued locally while the recipient is unreachable and replayed to it once a path opens, which is fair delivery (Section 2.11); the queue is on the sender and holds the sender's own messages.

No social graph in the layer. The layer recognizes the keys GLP supplies through `putKnownPeer` and `putPeerAddress` (Section 2.4). Addresses travel as GLP messages over authenticated sessions and enter the layer through `putPeerAddress`.

Dual-role BLE. Every BLE pair converges to two GATT legs, each device central on one leg and peripheral on the other. Platform asymmetries are met by choosing who initiates each leg: an iOS central cannot open the second link toward an Android it is already linked with, the connection wedging in `connecting` until timeout, so iOS opens a pair's first leg and the Android the reverse leg; iOS devices advertise the fixed local name `grs-ios` so that peers yield the first dial to them. Where a platform's behaviour is unknown, the leg is attempted and the hardware decides. A single-link pair is a transient state the layer keeps trying to upgrade, or one where the hardware has refused the second leg and the initiator order is the one lever left.

One address per peer. A connection uses one address pair for its lifetime. Each peer has one recorded WAN address, supplied by GLP through `putPeerAddress` or observed on a live session, the later replacing the earlier (Section 4.2); ANNOUNCE carries no address. The recorded address survives a stale-peer sweep, a disconnect on this side and a restart of a transport, since it is the last known location of the peer and what a reconnection attempt dials. `putPeerAddress` validates the address, creates the peer's entry even for a peer never seen, and supplies the key with it, so the peer joins the known-peer set; it is not reachable until a session authenticates. The agent's own interface addresses are read locally and are its host candidates, from which a dial picks the local end of its address pair. The agent asks no external service for its public address: the first connection to a declared rendezvous server seeds it from the address that server observes, later observations correct it, and `getPublicAddress` returns `null` until then (Section 4.2).

Transport independence. The PAN and the IP transports are independent: losing one has no effect on the other's connection state, the peer's reachability over it or the callbacks of Section 2.3, which fire per underlay. A peer reachable over the PAN and over IP fires two connects; losing one of two live underlays fires a disconnect for that underlay while the peer stays reachable over the other; `peerUnderlays` returns the underlays the peer is reachable over and `isPeerReachable` is the any-underlay predicate. The lifecycle of each transport service is `uninitialized`, `initializing`, `ready`, `active`, with `error` and `disposed`; a transport is usable when `ready` or `active`.

State. All peer and transport state of the Flutter package lives in one immutable Redux store, `AppState`: the discovered BLE devices and identified peers, the lifecycle and public address of each transport, the messages, the known-peer set with its recorded addresses, and the settings. Actions describe events and reducers produce the next state; there is no mutable singleton. The store is a projection of facts the transports emit: that a peer is gone is the transport's to report as an event, and a reducer does not conclude it from silence. The underlays the layer runs are fixed when it is instantiated (Section 2.2), and the demo application's switches are the application's choice of what to instantiate the layer with. The persisted settings carry a trust level per proximity underlay; a malformed settings record is discarded whole, leaving both levels Closed, which is what an unset level means. Text the demo application shows says "Internet", not the name of a protocol.

A.3 What Is Realized

The layer realizes the specification as follows. The five-byte frame of Section 2.10 on every packet of both carriers. The ANNOUNCE record of Section 2.5: key, version and trailing signature, a fixed 98-byte payload, any other length rejected, verified against the carried key; ANNOUNCE inside the session as the heartbeat, every 10 s, packet type `0x15`; a clear ANNOUNCE only before the XX handshake of a first contact under Open trust, and under Closed trust ignored and its connection dropped. Sessions as Section 2.6 states: Noise IK whenever the initiator expects the responder's key, XX only for an Open first contact after ANNOUNCE, the expected Curve25519 static computed from the peer's Ed25519 key; a Closed responder answers IK only from a known key, a rendezvous server from any key, and neither answers XX; the final handshake message retransmitted until the attestation message arrives. The attestation exchange of Section 2.6: the attestation once over the identity key and the per-session signature over the digest carrying the version number, each side's message as its own packet types after the final handshake message, reachability gated on the exchange, teardown on an attestation found invalid, nothing cached across sessions, an explicit statement of no attestation from a platform without one, and a build that cannot verify what it is offered reporting the peer unattested. The attested application as developer, name and version number (Section 2.6), `null` for an unattested identity. The message protocol of Section 2.10: self-contained session-encrypted fragments over UDP datagrams and GATT writes, per-fragment acknowledgement and retransmission with backoff under a bounded per-peer window, reassembly and atomic delivery, every reassembly delivered and acknowledged, so a message re-sent after a lost acknowledgement is delivered again; `send` held until the peer is reachable. Fair delivery (Section 2.11): every message `send` accepts is saved before `send` returns, removed on the recipient's acknowledgement, and read back and replayed when a layer with the same identity starts, the heartbeat tick driving the replay and the reconnection attempts to unreachable known peers and to the declared rendezvous servers; the Flutter profile keeps the queue in the platform's preferences store, one key per agent, and the headless profile in the file its embedder names; a delivered message is kept until `acknowledge` and delivered again after a restart. Proximity discovery (Section 3): the rotating recognition token in the BLE service UUID and as the DNS-SD instance name, re-advertised at each slot boundary and matched against the current and adjacent slots; `onPeerDiscovered` once per identity, for PAN discovery and LAN contact; a recognized presence contacted with IK, the

recognized key expected; the deterministic initiator by the smaller key; the two trust levels, both Closed until set, each governing its own underlay (Section 2.8). Inbound WAN contact answered only from a key in the known-peer set, or from any key on a rendezvous server, and LAN contact told from it by the source address against the agent's own local prefixes (Section 2.8). Address-change recovery as Section 4.2 states, both profiles sweeping their unreachable known peers and declared rendezvous servers on the heartbeat, and a session with each declared rendezvous server, established by the layer as with any peer and reported reachable, unattested, once established and attested. The rendezvous servers declared and removed by GLP through `putRendezvousServer` and `removeRendezvousServer` while the layer runs (Section 4.2). The local network fingerprint and its callback (Section 4.3), independent of the public address and its callback: neither refresh triggers the other. The place functions of Section 5 with the behaviour the specification states, including `unobservable` and `observable` on the stream of every standing declaration and a place declared while unobservable told so at once. `sign`, `verify`, `observedPeerAddress`, `punchUdp` and `peerUnderlays` on both profiles, `punchUdp` over the hole-punch service, carrying no payload; `observedPeerAddress` reports the address observed on the live session, not the recorded one. The protocol version is 8; a peer on an earlier version is refused.

A.4 Below the Specification

The following the implementation decides, and this appendix records as their definition.

The handshake. The handshake envelope is the version (2 bytes), the kind (IK `0x11` and `0x12`, XX `0x01` to `0x03`), for XX the identity claim and on message 1 its signature over the claim and the ephemeral key, then the Noise message; IK message 1 carries the initiator's Ed25519 key encrypted, checked against its static, and the prologue mixes in a label and the version. A dial runs IK at once, message 1 retransmitted at 0.8, 1.2, 2.4 and 4.8 s, up to five attempts; the final message is retransmitted from 0.8 s, doubling to 5 s, until the attestation message arrives; one handshake chain per medium; a retransmitted earlier message replays the stored later one; the two coordinators serialize the handshake state of a pair, glare resolved by the smaller key. A peripheral GATT leg under Open trust waits 1.5 s for the central's IK before a first-contact ANNOUNCE; a clear ANNOUNCE is answered once per connection. A verified ANNOUNCE from a new address resets a stale session. A new handshake against a live session waits beside it and replaces it when it completes: an IK handshake when the initiator's first packet opens under it, an XX handshake when its message 3 verifies; when either completes, a handshake of the other kind waiting is dropped, and a peer holds up to four sessions waiting to complete. The receiver waits for the peer's attestation message 30 s over UDP and 12 s over GATT, the span of the sender's attempts, then tears the session down; a second attestation on an attested session is ignored.

The attestation. The attestation field opens with a platform tag; the per-session signature is ECDSA P-256 over SHA-256; no App Attest counter is kept between sessions, the handshake hash already making each signature unique to its session; the Android producer offers no key held in software; a certificate that issues another must be a CA, which closes a forged Android chain under an application's own attestation key; an Android attestation

naming more than one package or more than one signing digest is invalid and its session torn down. The version number v of the digest is a whole number below 2^{63}, 8 bytes big-endian in the digest and in the message; on iOS it is `CFBundleVersion`, and a `CFBundleVersion` that is not a whole number gives no attestation, the peer reported unattested; on Android it is the version code. The developer is, on Android, the lowercase hexadecimal of the signing certificate's digest, and on iOS the team identifier, the developer and the name being the App ID split at its first dot. The networking demo application, development-signed, accepts App Attest's development sandbox, while the verifier's default is production only.

The packets. One framed packet per UDP datagram, within a path-MTU budget, chunks of about 1100 bytes over IP and about 270 over BLE. A connection is soft state, the peer's address and its Noise session, torn down by heartbeat silence (Section 2.5). The receive loop may drain several datagrams before asynchronous processing, so the pre-identification connection identifier is kept as an alias of the peer's address once the handshake or ANNOUNCE keys it; a pre-identification path is created only for the packet types that can open contact, ANNOUNCE and handshake, and unidentified paths are swept after brief silence, which bounds state under unsolicited traffic. Fragments carry `messageId`, index, count and chunk, reassembling from any subset in any arrival order; fragment acknowledgements carry `messageId` and index and drive retransmission with exponential backoff under a bounded per-peer send window filled round-robin across the peer's active messages; the round-robin decides who takes a freed slot and frees none, so a message whose fragments are all outstanding holds the window until its attempts are exhausted, and the attempt limit is what bounds one message's delay of another; a whole-message acknowledgement settles the message. The attestation exchange rides the protocol as the packet types `0x13` and `0x14`, outside `send` and the reachability-gated queue, one per session, checked before each transmission against the session being current. The punch datagrams of `punchUdp` open with the magic `GRPU` (`0x47 0x52 0x50 0x55`). Signalling packets pass between an agent and its declared rendezvous servers and carry the address the server observes the agent at.

The queue. The queue store is JSON with base64 payloads, the whole queue rewritten at each change, and holds the delivered messages not yet acknowledged beside the sent ones; a delivered message is saved before it is delivered and before the sender is acknowledged, and is delivered again when the queue is read and the receive callback set. Another identity's queue is refused, a queue without the field of delivered messages is malformed and refused, and an unreadable queue makes `start`, `initialize` and `send` throw. The identity's file form is `{"seed": <hex>}` and its map form `{publicKey, privateKey}`, any other refused. A declaration of a rendezvous server whose address changes keeps a standing session with it. The layer passes no BLE local name; the plugin `grassroots_bluetooth_layer` advertises the fixed local name `grs-ios` on iOS.

The local network. The fingerprint of Section 4.3 is the SHA-256 of the agent's local address prefixes under a domain separator, truncated to 16 bytes, the prefixes sorted and de-duplicated first, so neither the fingerprint nor the prefix list depends on the order the platform lists interfaces or on how many interface addresses share a subnet. Of the three

constituents the specification names, only the prefixes are readable: the platform's interface list carries no netmask, no default gateway and no wireless network name, and on iOS the wireless network name requires an entitlement. The prefix length is the conventional subnet size of the family, 24 bits for IPv4 and 64 for IPv6, so a wider real subnet is split into several prefixes and a peer elsewhere within it is not recognized as LAN contact, which is the safe direction for the trust decision of Section 2.8: the failure is a refused LAN peer, never an accepted stranger. Loopback is excluded; link-local addresses are included, a self-assigned network being a local network. An IPv4-mapped address is named in IPv4 form.

Places. Each declaration is registered under an identifier of its own, and the platform reports a crossing against that identifier rather than against the place name, which tells a crossing of an ended declaration from one of the standing declaration: removing a place and superseding it both drop the identifier, so a report that raced the unregistration finds nothing to deliver to and is dropped. A replacement unregisters the earlier geofence before registering the new one, so a replacement never exceeds the platform's cap on registered geofences; should the platform then refuse the new registration, the place is left undeclared. Disposal unregisters every standing declaration. A radius that is not a positive, finite number of metres is rejected. On iOS the layer evaluates crossings itself: region monitoring reports crossings to an application in the background and therefore needs the permission to use location at all times, which is not requested, so the binding registers the region and evaluates the foreground location against it, reporting the device unobservable whenever the application is not in the foreground (Section 2.7).

Cryptography. Ed25519 signing and verification use native libsodium; a pure-Dart implementation was measured at 150 to 200 ms per verification on Android phone hardware, which is prohibitive at one verification per packet, and remains relevant wherever GLP code verifies signatures on a phone.

A.5 Deviations and What Is Not Yet Implemented

- **madGLP runs over the layer headless, not yet on phones**: two processes, and the runtime's own hosts, two `AgentRuntime`s over `HeadlessGlpNetwork`; the on-device test bed was deleted with the demo it belonged to, and the super-app that replaced it has no two-phone build over the layer.
- **The attestation producers on a device**: the iOS producer runs on the simulator, where App Attest is unsupported, so key generation, attestation and the assertion have not run; the Android producer is written and has not been compiled, run or tested, the build machine having no JDK, no Android SDK and no device. The Swift producer parses and was not built.
- **The GATT path** of the handshake and the attestation is tested at the protocol level only, the layer taking no fake BLE host.

A.6 Testing on Two iPhones

The phones are an iPhone 16 Pro ("iUdi 16pro", iOS 26.5; flutter device `00008140-001C38C83A12801C`, devicectl `2219A3DC-78B5-5721-9E5E-FC2686D6A402`) and an iPhone 11 Pro Max ("iUdi 11pro", iOS 17.4.1; `00008030-000275600A12802E`, `C1F923A2-54D9-54DA-912D-8D033E7E425F`); the 11 Pro's provisioning, developer trust and Developer Mode are done. A build is installed with `xcrun devicectl device install app -device <UUID> <app>` and launched with `xcrun devicectl device process launch -device <UUID> <bundle id>`. What was learned: build and install the two roles serially, since concurrent `flutter build` invocations share the build directory and clobber each other's defines; iOS shows a Local Network permission dialog on first mDNS use, to be allowed on both phones; a WiFi network with client isolation blocks phone-to-phone traffic and mDNS multicast, so a pair stuck pairing for over a minute after the permission is the network, not the code, and a non-isolated LAN or a hotspot from a third device is used; stale simulator applications with the same seeded identities are killed before a test with a rendezvous server, or they poison its address table; a LAN instance unmatched with a plausible token is clock skew beyond the adjacent-slot match, and both phones' clocks are checked.